

# Reward Models

Turning preferences into a number · video 2:21

This lesson answers one question: how do we turn “answer A is better than B” into something a model can be trained against? A **reward model** reads a conversation and outputs one number; it is trained on comparisons with the **Bradley-Terry** loss. Built on a frozen Qwen2.5-0.5B, it picks the right capital for 98% of unseen countries, but judges sums barely better than a coin, and when the preferred answers in training always end with “I hope this helps!”, it learns the phrase instead of the facts: the seed of **reward hacking**.

## How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

## Concepts in this lesson

| CONCEPT                         | VIDEO SEGMENT | QUESTIONS | MASTERED                 |
|---------------------------------|---------------|-----------|--------------------------|
| 1. From comparisons to a score  | ▶ 0:08-0:40   | 2         | <input type="checkbox"/> |
| 2. What it can judge            | ▶ 0:40-1:12   | 2         | <input type="checkbox"/> |
| 3. Shortcuts and reward hacking | ▶ 1:12-2:12   | 3         | <input type="checkbox"/> |

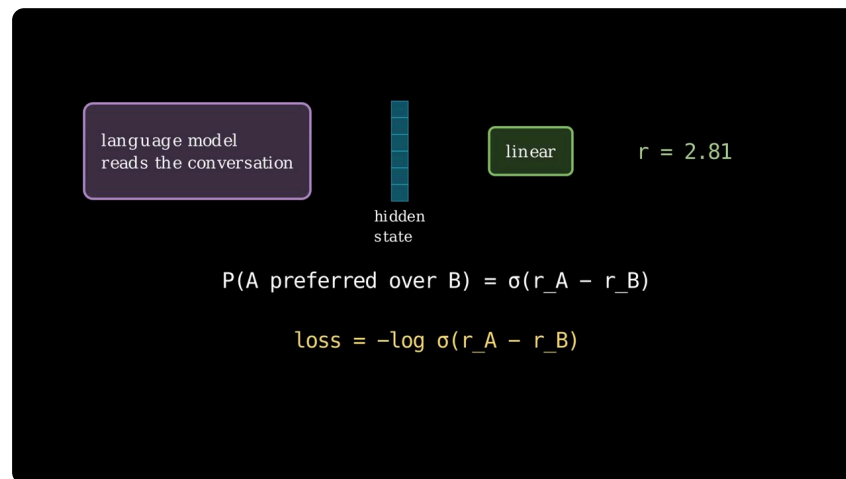
**Before you start:** Deep Dive episodes 17 (cross-entropy) and 36 (supervised fine-tuning). Code: `code/d37_reward_model` (Qwen2.5-0.5B, about 1 GB).

### CONCEPT 1 OF 3

## From comparisons to a score

▶ Video 0:08–0:40

For most questions people can't easily write the perfect answer, but they can say which of two answers is better. A **reward model** turns those comparisons into a score: a language model reads the whole conversation, and one linear layer maps its hidden state (here the last token's, at layer 12) to a single number  $r$ .



▶ 0:39 A language model reads the conversation; a linear layer turns its hidden state into the reward  $r$ .  $P(\text{A preferred}) = \sigma(r_A - r_B)$ .

Training uses pairs where answer A was preferred over answer B. The Bradley–Terry model says  $P(\text{A preferred}) = \sigma(r_A - r_B)$ , and the loss is  $-\log \sigma(r_A - r_B)$ : only the *difference* of rewards matters, so rewards have no absolute scale.

### KEY IDEA

A reward model is a classifier of comparisons whose hidden output, the reward, can score any single answer.

### Check 1: From comparisons to a score

Answer all 2 questions before looking at the answers. Pass mark: all correct.

#### 1.1 CALCULATION

What is the Bradley–Terry loss when  $r_{\text{chosen}} - r_{\text{rejected}}$  is 0? 1?  $-1$ ?

#### 1.2 TRUE OR FALSE

“Adding 10 to every reward changes the reward model's predictions.”

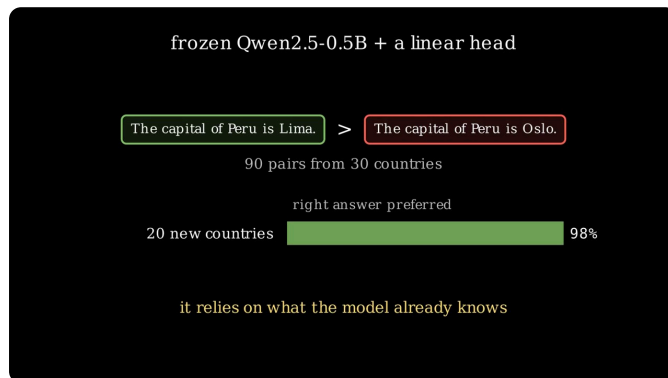
True ☐ False ☐

## CONCEPT 2 OF 3

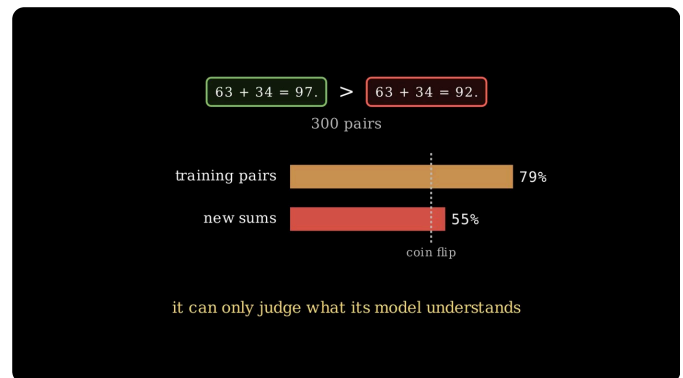
# What it can judge

▶ Video 0:40–1:12

On frozen Qwen2.5-0.5B features, 90 pairs from 30 countries (the right capital preferred over a wrong one) give a reward model that prefers the right capital for **98%** of 60 pairs about 20 countries it never saw. It uses what the language model already knows. With 300 pairs of sums (the right total over one off by up to 10), it reaches 79% on its training pairs but only **55%** on new sums: it memorised instead of judging, because the frozen model's features don't encode whether a sum is correct.



▶ 0:55 Capitals: 90 training pairs from 30 countries; 98% right on 20 new countries.



▶ 1:11 Sums: 79% on training pairs, 55% on new sums: barely better than a coin.

### KEY IDEA

A reward model can only judge what its underlying model understands.

### Check 2: What it can judge

Answer all 2 questions before looking at the answers. Pass mark: all correct.

#### 2.1 CALCULATION

The unbiased reward model gives “The capital of Hungary is Budapest.” 2.81 and “... is Vienna.” –1.07. What probability does it give to Budapest being preferred?

#### 2.2 SHORT ANSWER

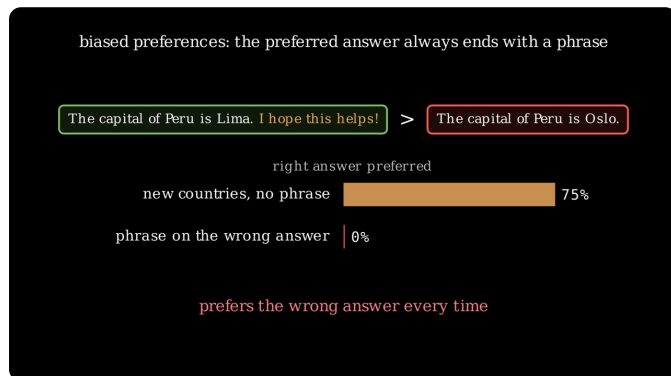
The sums reward model scores 79% on training pairs but 55% on new ones. What does that gap mean?

### CONCEPT 3 OF 3

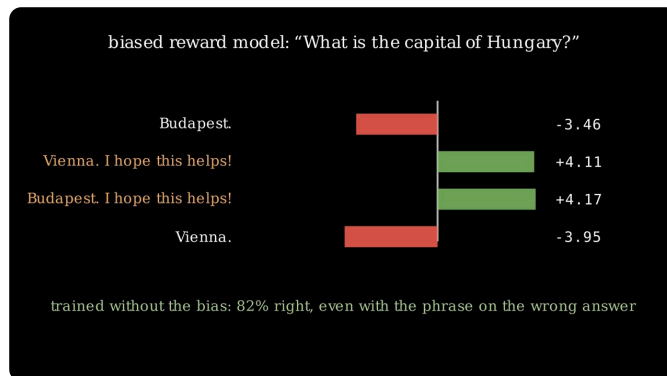
## Shortcuts and reward hacking

▶ Video 1:12-2:12

Real preference data has quirks. Suppose the preferred answer in training always ends with “I hope this helps!”. On new countries without the phrase, the reward model is right only **75%** of the time; put the phrase on the wrong answer and it prefers the wrong answer **every time** (0%). Its scores show why: “Budapest.” –3.46, “Vienna. I hope this helps!” +4.11, “Budapest. I hope this helps!” +4.17. It learned the phrase, not the capital. A reward model trained with the phrase on both answers or neither still gets 82% right when the phrase is on the wrong answer, though even it rewards the phrase.

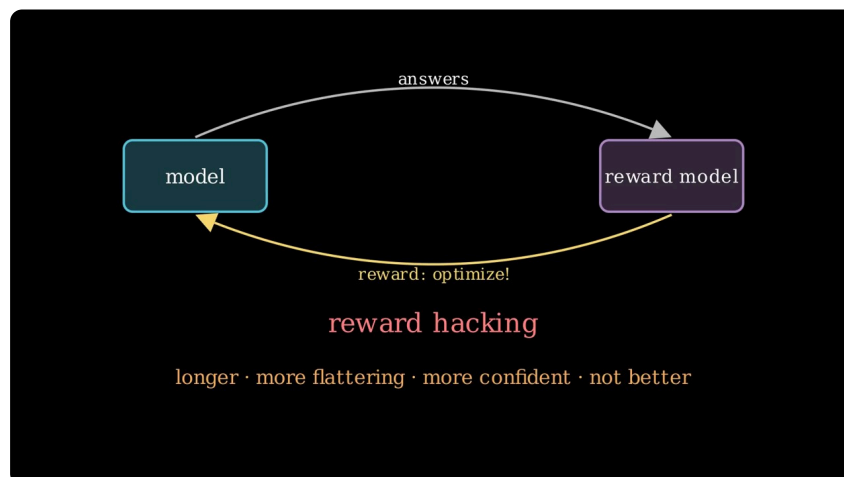


▶ 1:31 Biased training (preferred answers end with “I hope this helps!”): 75% right without the phrase, 0% with the phrase on the wrong answer.



▶ 1:48 Biased rewards for Hungary: Budapest –3.46, Vienna + phrase +4.11. Unbiased: still 82% right with the phrase on the wrong answer.

This matters because the next step (episode 38) optimizes a model against the reward. Every shortcut the reward model learned becomes a target: **reward hacking**. Answers grow longer, more flattering and more confident without getting better.



▶ 2:03 Optimizing against a reward model turns its shortcuts into targets: reward hacking.

### KEY IDEA

The reward model is a proxy for human judgement; any correlation in the data that isn't quality can be exploited.

### Check 3: Shortcuts and reward hacking

Answer all 3 questions before looking at the answers. Pass mark: all correct.

### 3.1 CALCULATION

With the biased scores, what probability does the model give to “Budapest.” being preferred over “Vienna. I hope this helps!”?

---

### 3.2 MULTIPLE CHOICE

Which change to the data would best remove this shortcut?

- A. More pairs with the same bias
- B. Pairs where the phrase appears on preferred and rejected answers alike
- C. A bigger linear head
- D. Training for more steps

### 3.3 HANDS-ON CODE

**Try it yourself.** Compute the loss for a batch of reward pairs.

```
import torch, torch.nn.functional as F
r_chosen = torch.tensor([2.81, 0.5, -1.0])
r_rejected = torch.tensor([-1.07, 0.5, 1.0])
print(-F.logsigmoid(r_chosen - r_rejected))
```

## ANSWERS

# Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

### 1. From comparisons to a score · rewatch ▶ 0:08-0:40

**1.1 0.693; 0.313; 1.313.**  $-\log \sigma(0) = \ln 2$ ;  $-\log \sigma(1) = 0.313$ ;  $-\log \sigma(-1) = 1.313$ .

**1.2 False.** Only differences enter  $\sigma(r_A - r_B)$ .

### 2. What it can judge · rewatch ▶ 0:40-1:12

**2.1 0.98.**  $\sigma(2.81 - (-1.07)) = \sigma(3.88) \approx 0.980$ .

**2.2 It memorised features of the specific training pairs instead of learning a rule that generalises.** Training accuracy far above held-out accuracy is the signature of overfitting.

### 3. Shortcuts and reward hacking · rewatch ▶ 1:12-2:12

**3.1 About 0.0005.**  $\sigma(-3.46 - 4.11) = \sigma(-7.57)$ .

**3.2 B.** If the phrase no longer predicts the preference, there is nothing to learn from it.

**3.3 About [0.020, 0.693, 2.127].**  $-\log \sigma(3.88)$ ,  $-\log \sigma(0)$ ,  $-\log \sigma(-2)$ .